

数字电路设计

DDCA 第 7 章详细学习笔记

Digital Design and Computer Architecture: ARM[®] Edition
Sarah L. Harris & David Money Harris

主题：微体系结构（Microarchitecture）

涵盖：单周期 • 多周期 • 流水线 • 高级微体系结构

总页数：194 页

生成日期：2026 年 6 月 15 日

目录

1 引言与性能分析 (Introduction & Performance)	3
1.1 微体系结构概述	3
1.2 三种微体系结构实现	3
1.3 处理器性能公式	3
1.4 本书考虑的 ARM 指令子集	4
1.5 架构状态元素	4
2 单周期 ARM 处理器 (Single-Cycle ARM Processor)	5
2.1 单周期数据通路构建	5
2.1.1 LDR 指令对应的数据通路	5
2.1.2 PC 的访问	5
2.1.3 STR 指令的扩展	6
2.1.4 数据处理指令的支持	6
2.1.5 B 指令的支持	6
2.2 单周期控制器 (Single-Cycle Control)	6
2.2.1 控制信号分类	7
2.2.2 FlagW 信号	7
2.2.3 解码器子模块 (Decoder Submodules)	7
2.2.4 主解码器 (Main Decoder)	8
2.2.5 ALU 回顾与 ALU 解码器	8
2.2.6 PC 逻辑	9
2.2.7 条件逻辑 (Conditional Logic)	9
2.2.8 条件执行 (Conditional Execution)	9
2.2.9 标志位更新 (Set Flags)	10
2.2.10 扩展功能: CMP 指令	11
2.2.11 扩展功能: 移位寄存器	11
2.3 单周期处理器性能分析	12
2.3.1 关键路径 (Critical Path)	12
2.3.2 单周期性能示例	12

3 多周期 ARM 处理器 (Multicycle ARM Processor)	13
3.1 为什么需要多周期	13
3.2 多周期数据通路	13
3.2.1 统一存储器	13
3.2.2 LDR 指令执行过程 (5 个周期)	13
3.2.3 PC 的读写访问	14
3.2.4 STR 指令	14
3.2.5 数据处理指令	14
3.2.6 B 指令	15
3.3 多周期控制器 (Multicycle Control)	15
3.3.1 指令解码器	15
3.3.2 主控制器有限状态机 (Main FSM)	15
3.3.3 各指令周期数	16
3.3.4 多周期条件逻辑	16
3.4 多周期处理器性能分析	16
3.4.1 平均 CPI 计算	16
3.4.2 关键路径	17
3.4.3 执行时间	17
4 流水线 ARM 处理器 (Pipelined ARM Processor)	18
4.1 流水线原理	18
4.1.1 五级流水线	18
4.1.2 流水线数据通路优化	18
4.2 流水线控制器	18
4.3 流水线冒险 (Pipeline Hazards)	19
4.3.1 处理数据冒险的四种方法	19
4.3.2 数据前推 (Data Forwarding)	19
4.3.3 停顿 (Stalling)	20
4.3.4 控制冒险 (Control Hazards)	20
4.3.5 提前分支解析 (Early Branch Resolution)	21
4.3.6 控制停顿逻辑	21

4.4	流水线处理器性能分析	21
4.4.1	流水线 CPI 计算示例	21
4.4.2	流水线关键路径	22
4.4.3	流水线执行时间	22
4.4.4	三种处理器性能对比	23
5	高级微体系结构 (Advanced Microarchitecture)	24
5.1	深度流水线 (Deep Pipelining)	24
5.2	微操作 (Micro-operations)	24
5.3	分支预测 (Branch Prediction)	24
5.3.1	静态分支预测	25
5.3.2	动态分支预测	25
5.4	超标量处理器 (Superscalar)	25
5.5	乱序处理器 (Out of Order Processor)	26
5.6	寄存器重命名 (Register Renaming)	27
5.7	SIMD (单指令多数据)	27
5.8	多线程 (Multithreading)	27
5.8.1	线程定义	27
5.8.2	传统处理器中的线程	28
5.8.3	硬件多线程	28
5.9	多处理器 (Multiprocessors)	28
5.10	延伸阅读资源	29
6	核心公式速查表 (Key Formula Quick Reference)	30
7	三种实现对比总结	30
8	考点指导 (Study Guidance)	30
8.1	高频考点	30
8.2	中等考点	31
8.3	了解性考点	31

1 引言与性能分析 (Introduction & Performance)

1.1 微体系结构概述

[p.3–4]

- **微体系结构 (Microarchitecture)**

微体系结构 (Microarchitecture) 指的是如何在硬件上实现一个指令集架构 (Architecture)。一个架构可以有多种不同的微体系结构实现。[p.3–4]

一个处理器由两大部分组成:

- **数据通路 (Datapath):** 功能模块 (functional blocks), 如 ALU、寄存器文件、存储器等。[p.3]
- **控制器 (Control):** 产生控制信号以协调数据通路各模块的工作。[p.3]

1.2 三种微体系结构实现

[p.4]

对于同一架构, 可以通过三种不同的微体系结构实现:

- **单周期 (Single-Cycle):** 每条指令在一个时钟周期内执行完成。
- **多周期 (Multicycle):** 每条指令被拆分成一系列较短的步骤 (每个步骤一个周期)。
- **流水线 (Pipelined):** 指令被拆分为多个阶段 (stage), 同时有多条指令在不同阶段执行。

三种实现代表了性能-复杂度的不同折中。[p.4]

1.3 处理器性能公式

[p.5]

- **执行时间公式**

核心性能公式:

$$\text{Execution Time} = (\# \text{instructions}) \times (\text{cycles/instruction}) \times (\text{seconds/cycle})$$

- **CPI (Cycles Per Instruction):** 每条指令的平均周期数。[p.5]

- **时钟周期 (Clock Period):** T_c , 每个时钟周期的秒数。 [p.5]
- **IPC (Instructions Per Cycle):** $IPC = 1/CPI$ 。 [p.5]

设计的核心约束：**成本 (Cost)**、**功耗 (Power)**、**性能 (Performance)** 三者之间的权衡。 [p.5]

1.4 本书考虑的 ARM 指令子集

[p.6]

为简化分析，本章考虑 ARM 指令集的一个子集：

- **数据处理指令 (Data-processing):** ADD, SUB, AND, ORR (含寄存器和立即数 Src2, 不考虑移位)。 [p.6]
- **访存指令 (Memory):** LDR (加载), STR (存储), 使用正立即数偏移 (positive immediate offset)。 [p.6]
- **分支指令 (Branch):** B。 [p.6]

1.5 架构状态元素

[p.7]

决定处理器所有行为的关键状态元素：**16 个寄存器** (R0–R15, 其中 R15 = PC) 和**状态寄存器 (Status Register)**, 以及**存储器 (Memory)**。 [p.7]

2 单周期 ARM 处理器 (Single-Cycle ARM Processor)

2.1 单周期数据通路构建

[p.8–28]

单周期处理器的设计遵循两步法：先设计数据通路 (Datapath)，再设计控制器 (Control)。

[p.9–10]

2.1.1 LDR 指令对应的数据通路

[p.11–17]

以 `LDR Rd, [Rn, #imm12]` 为例，单周期数据通路按以下 6 步构建：

1. **取指 (Fetch)**：从指令存储器 (Instruction Memory) 中读取指令，PC 指向当前指令。[p.12]
2. **读寄存器 (Register Read)**：从寄存器文件 (Register File, RF) 中读取源操作数 R_n 。[p.13]
3. **立即数扩展 (Immediate Extension)**：将 12 位立即数扩展为 32 位。[p.14]
4. **计算内存地址 (Address Computation)**：ALU 将 R_n 与扩展后的立即数相加，得到内存地址。[p.15]
5. **读内存并写回**：从数据存储器 (Data Memory) 读出数据，写回寄存器文件的目标寄存器 R_d 。[p.16]
6. **更新 PC (PC Increment)**：确定下一条指令的地址 ($PC+4$)。[p.17]

2.1.2 PC 的访问

[p.18–20]

PC (R_{15}) 既可以作为源操作数也可以作为目标操作数：

- **作为源 (Source)**： R_{15} 必须在寄存器文件中可用，读取 PC 时返回当前 $PC + 8$ (由于 ARM 三级流水线架构遗留)。[p.19]
- **作为目标 (Destination)**：必须能够将计算结果写回 PC 寄存器。[p.20]

2.1.3 STR 指令的扩展

[p.21]

为支持 STR Rd, [Rn, #imm12], 需要在数据通路中新增: 将 Rd 的数据写入数据存储器。与 LDR 不同, STR 需要: 读取 Rd 的值 (作为写入存储器的数据), 同时 ALU 计算目标地址。[p.21]

2.1.4 数据处理指令的支持

[p.22–25]

立即数寻址 (Immediate Src2): 如 ADD Rd, Rn, #imm8

- 从 Rn 和 Imm8 读取操作数 (ImmSrc 选择零扩展-zero-extended 8 位立即数, 而非 Imm12)。[p.22–23]
- ALUResult 写入寄存器文件的 Rd。[p.22–23]

寄存器寻址 (Register Src2): 如 ADD Rd, Rn, Rm

- 从 Rn 和 Rm 读取操作数 (而非 Imm8)。[p.24–25]
- ALUResult 写入寄存器文件的 Rd。[p.24–25]

2.1.5 B 指令的支持

[p.26–27]

分支目标地址 (Branch Target Address, BTA) 的计算公式:

$$\text{BTA} = \text{ExtImm} + (\text{PC} + 8)$$

其中 $\text{ExtImm} = \text{Imm24} \ll 2$ 并做符号扩展 (sign-extended)。[p.26]

立即数扩展控制 (ImmSrc): [p.27]

ImmSrc[1:0]	ExtImm	描述
00	{24'b0, Instr[7:0]}	零扩展 8 位立即数
01	{20'b0, Instr[11:0]}	零扩展 12 位立即数
10	{6{Instr[23]}, Instr[23:0]}	符号扩展 24 位立即数

2.2 单周期控制器 (Single-Cycle Control)

[p.29–71]

2.2.1 控制信号分类

[p.30–32]

控制信号分为两类：

- 直接发送到数据通路的信号：ALU 控制、多路选择器选择信号、ImmSrc 等。[p.30]
- 经过条件逻辑 (Conditional Logic) 后再发送的信号：PCSrc、RegWrite、MemWrite 等改变处理器状态 (PC、寄存文件、存储器) 的信号。[p.31–32]

条件执行：如果指令不应执行 (条件不满足)，这些状态修改信号被强制为 0，从而阻止状态改变。[p.32]

2.2.2 FlagW 信号

[p.33–35]

FlagW[1:0] 是标志写使能信号。当指令的 S 位 (Suffix S) 为 1 时，ALU 产生的标志位 (Flags) 应被保存到状态寄存器。

- ADD, SUB (带 S 后缀)：更新全部 4 个标志位 NZCV。[p.34]
- AND, ORR (带 S 后缀)：只更新 N 和 Z 标志位 (C 和 V 不受逻辑运算影响)。[p.34]
- 因此需要两个标志写使能位：[p.35]
 - FlagW[1] = 1：N, Z (ALUFlags[3:2]) 被保存
 - FlagW[0] = 1：C, V (ALUFlags[1:0]) 被保存

2.2.3 解码器子模块 (Decoder Submodules)

[p.37–41]

解码器由三个子模块组成：[p.38–39]

1. 主解码器 (Main Decoder)
2. ALU 解码器 (ALU Decoder)
3. PC 逻辑 (PC Logic)

2.2.4 主解码器 (Main Decoder)

[p.40]

主解码器根据指令的操作码 (Op)、Funct5 和 Funct0 生成主要的控制信号：

Op	F5	F0	类型	Branch	MemtoReg	MemW	ALUSrc	ImmSrc	RegW	RegSrc
00	0	X	DP Reg	0	0	0	0	XX	1	00
00	1	X	DP Imm	0	0	0	1	00	1	X0
01	X	0	STR	0	X	1	1	01	0	10
01	X	1	LDR	0	1	0	1	01	1	X0
11	X	X	B	1	0	0	1	10	0	X1

字段说明：

- **MemtoReg**: 选择写回寄存器文件的数据来源 (0 = ALUResult, 1 = 从内存读取的数据)。[p.40]
- **MemW (MemWrite)**: 使能数据存储器写操作。[p.40]
- **ALUSrc**: 选择 ALU 的第二个源操作数 (0 = 寄存器, 1 = 立即数)。[p.40]
- **ImmSrc**: 选择立即数扩展方式 (00/01/10 对应上述三种方式)。[p.40]
- **RegW (RegWrite)**: 使能寄存器文件写操作。[p.40]
- **RegSrc**: 选择寄存器文件的读地址来源。[p.40]
- **ALUOp**: 指示这是否为数据处理指令。1 = 是 (ALU 控制由 funct 域决定), 0 = 否 (ALU 做加法)。[p.40]

2.2.5 ALU 回顾与 ALU 解码器

[p.42–45]

ALU 的基本功能 (ALUControl[1:0]): [p.42]

ALUControl[1:0]	功能
00	ADD (加法)
01	SUB (减法)
10	AND
11	ORR

ALU 解码器真值表: [p.45]

ALUOp	Funct[4:1]	Funct[0]	类型	ALUCtrl[1:0]	FlagW[1:0]
0	X	X	Not DP	00	00
1	0100	0	ADD	00	00
		1			11
1	0010	0	SUB	01	00
		1			11
1	0000	0	AND	10	00
		1			10
1	1100	0	ORR	11	00
		1			10

注意: 非 DP 指令 (ALUOp=0) FlagW=00, 即不更新标志位。当 S 位 =1 时, FlagW 根据指令类型设为 11 或 10。[p.45]

2.2.6 PC 逻辑

[p.47]

PCSrc (PC 写使能) 的逻辑:

$$PCSrc = ((Rd == 15) \wedge RegW) \vee Branch$$

即: 如果目的寄存器是 R15 (PC) 且 RegW 有效 **或者** 当前指令是分支指令, 则 PCS=1。最终 PCSrc 由条件逻辑门控: 若指令被执行则 PCSrc = PCS, 否则 PCSrc = 0 (即 PC = PC + 4)。[p.47]

2.2.7 条件逻辑 (Conditional Logic)

[p.49–63]

条件逻辑有两个功能: [p.50–51]

1. 检查指令是否应该执行: 如果不执行, 将 PCSrc、RegWrite 和 MemWrite 强制为 0。
[p.50]
2. 可能更新状态寄存器 (Flags[3:0] = NZCV)。[p.50]

2.2.8 条件执行 (Conditional Execution)

[p.53–57]

根据条件助记符 (Cond[3:0]) 和当前标志位 (Flags[3:0]), 产生 CondEx 信号 (CondEx = 1 表示指令应被执行)。[p.53–54]

ARM 条件码总结: [p.55]

Cond[3:0]	助记符	含义
0000	EQ	相等 (Z=1)
0001	NE	不等 (Z=0)
0010	CS/HS	进位设置 / 无符号大于或等于 (C=1)
0011	CC/LO	进位清零 / 无符号小于 (C=0)
0100	MI	负 / 负数 (N=1)
0101	PL	正 / 正数或零 (N=0)
0110	VS	溢出设置 (V=1)
0111	VC	无溢出 (V=0)
1000	HI	无符号大于 (C=1 且 Z=0)
1001	LS	无符号小于或等于 (C=0 或 Z=1)
1010	GE	有符号大于或等于 (N=V)
1011	LT	有符号小于 (N≠V)
1100	GT	有符号大于 (Z=0 且 N=V)
1101	LE	有符号小于或等于 (Z=1 或 N≠V)
1110	AL	总是 / 无条件

示例:

- AND R1, R2, R3: Cond = 1110 (AL) $\Rightarrow$ CondEx = 1。[p.56]
- EOREQ R5, R6, R7: Cond = 0000 (EQ), 若 Flags[3:2]=01 (Z=1) 则 CondEx = 1。
[p.57]

2.2.9 标志位更新 (Set Flags)

[p.59–63]

Flags[3:0] = NZCV 的更新条件:

- FlagW[1:0] 非零 (即指令的 S 位 =1) 且
- CondEx = 1 (指令应该被执行)

由于 ADD/SUB 更新全部 4 个标志位而 AND/ORR 只更新 N 和 Z, 状态寄存器有两个独立的写使能: [p.60]

- FlagW[1]: 控制 NZ (Flags[3:2]) 的写入
- FlagW[0]: 控制 CV (Flags[1:0]) 的写入

示例: [p.62–63]

- SUBS R5, R6, R7: FlagW=11, CondEx=1 $\Rightarrow$ FlagWrite=11, 更新全部 4 个标志位。
[p.62]
- ANDS R7, R1, R3: FlagW=10, CondEx=1 $\Rightarrow$ FlagWrite=10, 仅更新 NZ。 [p.63]

2.2.10 扩展功能: CMP 指令

[p.66–69]

CMP (Compare) 指令实际上是带 S 后缀且 Rd 被忽略的 SUB 指令 (即 SUBS R0, Rn, Rm 的效果, 但不写回结果)。 [p.66–67]

数据通路无需任何修改。只需在 ALU 解码器中增加一条记录: [p.68–69]

ALUOp	Funct[4:1]	Funct[0]	类型	ALUCtrl	FlagW	NoWrite
1	1010	1	CMP	01	11	1

NoWrite=1 表示不将 ALU 结果写回寄存器文件, 仅更新标志位。 [p.69]

2.2.11 扩展功能: 移位寄存器

[p.70–71]

带移位操作的寄存器寻址 (shifted register) – 仅需在数据通路中添加移位器 (Shifter), 控制器无需修改。 [p.70–71]

2.3 单周期处理器性能分析

[p.72–77]

2.3.1 关键路径 (Critical Path)

[p.73–74]

单周期处理器的时钟周期 T_{c1} 受最慢指令（LDR）的关键路径限制：[p.73]

$$T_{c1} = t_{pcq_PC} + t_{mem} + t_{dec} + \max[t_{mux} + t_{RFread}, t_{sext} + t_{mux}] \\ + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

简化后的典型关键路径：

$$T_{c1} = t_{pcq_PC} + 2t_{mem} + t_{dec} + t_{RFread} + t_{ALU} + 2t_{mux} + t_{RFsetup}$$

关键路径主要由存储器、ALU 和寄存器文件的延迟决定。[p.74]

2.3.2 单周期性能示例

[p.75–77]

元件 (Element)	参数 (Parameter)	延迟 (ps)
寄存器 clock-to-Q	t_{pcq_PC}	40
寄存器 setup	t_{setup}	50
多路选择器	t_{mux}	25
ALU	t_{ALU}	120
解码器	t_{dec}	70
存储器读	t_{mem}	200
寄存器文件读	t_{RFread}	100
寄存器文件 setup	$t_{RFsetup}$	60

$$T_{c1} = 40 + 2(200) + 70 + 100 + 120 + 2(25) + 60 = \mathbf{840 \text{ ps}}$$

对于 1000 亿条指令的程序：[p.77]

$$\text{Execution Time} = (100 \times 10^9) \times 1 \times (840 \times 10^{-12}) = \mathbf{84 \text{ 秒}}$$

3 多周期 ARM 处理器 (Multicycle ARM Processor)

3.1 为什么需要多周期

[p.78–80]

单周期处理器的三个主要问题: [p.78]

- 时钟周期受最慢指令 (LDR) 限制—简单指令被迫等待。[p.78]
- 需要独立的指令存储器和数据存储器 (Harvard 架构, 不太现实)。[p.78]
- 需要 3 个加法器/ALU—硬件冗余。[p.78]

多周期处理器的优势: [p.79–80]

- 更高的时钟频率 (更快的关键路径)。[p.79]
- 更简单的指令运行更快 (不浪费周期)。[p.79]
- 重复使用昂贵的硬件 (多个周期共享同一功能单元)。[p.79]
- 缺点: 每个指令都要付出时序开销 (sequencing overhead)。[p.79]

与单周期相同的设计步骤: 先设计数据通路, 再设计控制器。[p.80]

3.2 多周期数据通路

[p.81–102]

3.2.1 统一存储器

[p.81]

将单周期的独立指令存储器和数据存储器替换为单一统一存储器 (single unified memory), 更加贴近实际硬件。[p.81]

3.2.2 LDR 指令执行过程 (5 个周期)

[p.82–87]

1. 取指 (Fetch): 从统一存储器读取指令。[p.82]
2. 读寄存器 (Register Read): 从寄存器文件读取 R_n 。[p.83]

3. **计算地址 (Address Computation):** ALU 计算 $R_n + \text{imm12}$ 。[p.84]

4. **读内存 (Memory Read):** 从统一存储器读取数据。[p.85]

5. **写回寄存器 (Write Back):** 将数据写入寄存器文件的 R_d 。[p.86]

第 6 步 **更新 PC** ($PC = PC + 4$) 在取指阶段同时完成。[p.87]

3.2.3 PC 的读写访问

[p.88–97]

读 PC (R15 作为源) [p.89–93]

例: `ADD R1, R15, R2`。R15 在寄存器文件中需要被读取为 $PC + 8$ 。在第 2 步 (读寄存器阶段), ALU 同时计算 $PC + 8$:

- $\text{SrcA} = PC$ (在第一步已更新为 $PC + 4$)。[p.92]
- $\text{SrcB} = 4$ 。[p.92]
- $\text{ALUResult} = PC + 8$ 送入 R15 的 RF 输入端口, 然后被路由至 RF 的 RD1 输出。
[p.92]

写 PC (R15 作为目标) [p.94–97]

例: `SUB R15, R8, R3`。指令的结果需要写入 PC 寄存器。由于 ALUResult 已经连接到 PC 寄存器的输入端, 只需使能 PCWrite 即可。[p.96–97]

3.2.4 STR 指令

[p.98]

为支持 STR, 需要在数据通路中将 R_n 的值写入统一存储器。与 LDR 的区别在于存储器写使能信号。[p.98]

3.2.5 数据处理指令

[p.99–100]

- **立即数寻址:** 数据通路无需额外修改, ImmSrc 选择立即数扩展方式即可。[p.99]
- **寄存器寻址:** 从 R_n 和 R_m 读取操作数。[p.100]

3.2.6 B 指令

[p.101]

分支目标地址计算公式与单周期相同：

$$\text{BTA} = \text{ExtImm} + (\text{PC} + 8)$$

其中 $\text{ExtImm} = \text{Imm24} \ll 2$ 并符号扩展。[p.101]

3.3 多周期控制器 (Multicycle Control)

[p.103–123]

3.3.1 指令解码器

[p.104–107]

ALU 解码器和 PC 逻辑与单周期相同。[p.106]

指令解码器真值表：[p.107]

指令	Op	Funct5	Funct0	RegSrc[0]	RegSrc[1]	ImmSrc[1:0]
LDR	01	X	1	0	X	01
STR	01	X	0	0	1	01
DP immediate	00	1	X	0	X	00
DP register	00	0	X	0	0	00
B	10	X	X	1	X	10

3.3.2 主控制器有限状态机 (Main FSM)

[p.109–119]

多周期控制器的核心是一个有限状态机 (FSM)，定义了每条指令的状态序列。[p.109]

FSM 状态：[p.110–118]

- **Fetch (S0)**: 从统一存储器取指令， $\text{PC} = \text{PC} + 4$ 。所有指令的起始状态。[p.110]
- **Decode (S1)**: 读寄存器文件，根据指令类型计算 $\text{PC} + 8$ ，解码确定下一个状态。
[p.111]
- **MemAdr (S2)**: 计算内存地址 ($\text{Rn} + \text{imm12}$)，用于 LDR/STR。[p.112]
- **MemRead (S3)**: 从存储器读取数据 (LDR)。[p.113]

- **MemWrite (S3')**: 将数据写入存储器 (STR)。[p.116]
- **MemWB (S4)**: 将读出的内存数据写回寄存器文件 (LDR 完成)。[p.115]
- **ExecuteI (S2')**: 执行立即数 DP 指令, 写回结果 (DP 完成)。[p.117]
- **ExecuteR (S2'')**: 执行寄存器 DP 指令, 写回结果 (DP 完成)。[p.118]

3.3.3 各指令周期数

[p.124–127]

不同指令需要的周期数不同: [p.127]

周期数	指令类型
3	B
4	DP (数据处理), STR
5	LDR

3.3.4 多周期条件逻辑

[p.120–123]

与单周期条件逻辑的关键区别: [p.123]

- **Fetch 状态**: PCWrite 被使能 (PC 在每条指令开始时总是被更新)。[p.123]
- **ExecuteI/ExecuteR 状态**: CondEx 使能, ALUFlags 产生。[p.123]
- **ALUWB 状态**: Flags 更新, CondEx 改变。PCWrite、RegWrite、MemWrite 在 Fetch 状态之前不会看到这个改变。[p.123]

关键点: 在单周期中, 条件逻辑直接门控写使能信号; 在多周期中, 条件逻辑的输出影响后续状态的行为, 但由于时钟边界的存在, 这种影响在下一个状态才会体现。

3.4 多周期处理器性能分析

[p.124–135]

3.4.1 平均 CPI 计算

[p.128–129]

使用 SPECINT2000 基准程序的指令混合比例: [p.128]

- 25% loads (LDR) → 5 周期
- 10% stores (STR) → 4 周期
- 13% branches (B) → 3 周期
- 52% DP (R-type) → 4 周期

$$\text{Average CPI} = (0.13)(3) + (0.52 + 0.10)(4) + (0.25)(5) = \mathbf{4.12}$$

3.4.2 关键路径

[p.130–132]

多周期的关键路径（假设 RF 比内存快，写内存比读内存快）：[p.130]

$$T_{c2} = t_{pcq} + 2t_{mux} + \max[t_{ALU} + t_{mux}, t_{mem}] + t_{setup}$$

使用相同的延迟参数：[p.131–132]

$$T_{c2} = 40 + 2(25) + \max[120 + 25, 200] + 50 = 40 + 50 + 200 + 50 = \mathbf{340 \text{ ps}}$$

3.4.3 执行时间

[p.133–135]

对于 1000 亿条指令的程序：

$$\begin{aligned} \text{Execution Time} &= (100 \times 10^9)(4.12)(340 \times 10^{-12}) \\ &= \mathbf{140 \text{ 秒}} \end{aligned}$$

这比单周期的 84 秒更慢！因为 CPI 从 1 增加到 4.12，虽然时钟周期从 840ps 降到 340ps（约 2.5 倍加速），但 CPI 的增加（4.12 倍）抵消了时钟频率的提升。[p.135]

4 流水线 ARM 处理器 (Pipelined ARM Processor)

4.1 流水线原理

[p.138–143]

4.1.1 五级流水线

[p.138]

流水线利用**时间并行性 (temporal parallelism)**，将单周期处理器的执行过程分为 5 个阶段 (stage)：[p.138]

1. **Fetch (F)**: 取指
2. **Decode (D)**: 译码
3. **Execute (E)**: 执行
4. **Memory (M)**: 访存
5. **Writeback (W)**: 写回

在各阶段之间插入**流水线寄存器 (Pipeline Registers)** 来分隔各阶段的数据。[p.138]

4.1.2 流水线数据通路优化

[p.141–143]

与单周期数据通路相比的改进：

- **修正的数据通路**：WA3（写地址）必须与 Result（写数据）同时到达寄存器文件；寄存器文件在 CLK 下降沿写入。[p.142]
- **优化**：通过使用 PCPlus4F（PC+4 已经存储在流水线寄存器中）来移除额外加法器。[p.143]

4.2 流水线控制器

[p.144]

流水线的控制单元与单周期处理器相同，但控制信号需要被延迟（随流水线寄存器传递）到正确的流水线阶段。每个流水线寄存器不仅传递数据，还传递对应的控制信号。
[p.144]

4.3 流水线冒险 (Pipeline Hazards)

[p.145–163]

当一条指令依赖于尚未完成的指令的结果时，就会发生冒险。[p.145]

两种类型：

- **数据冒险 (Data Hazard)**：寄存器值尚未写回寄存器文件。[p.145]
- **控制冒险 (Control Hazard)**：下一条指令尚未确定（由分支引起）。[p.145]

4.3.1 处理数据冒险的四种方法

[p.147–157]

1. **编译时插入 NOP**：在代码中插入足够的 NOP 指令，等待结果就绪。[p.147–148]
2. **编译时重排代码**：将不依赖的后续指令移到前面填充延迟槽。[p.147–148]
3. **运行时数据前推 (Data Forwarding)**：直接将前一条指令的结果（在流水线寄存器中）转发给后一条指令，无需等待写回寄存器文件。[p.147, 149–153]
4. **运行时停顿 (Stalling)**：硬件检测到数据依赖时暂停流水线，等待结果就绪。[p.147, 154–157]

4.3.2 数据前推 (Data Forwarding)

[p.149–153]

前推的核心逻辑（针对 Execute 阶段的源寄存器）：[p.152]

- **检查 Execute 阶段寄存器是否匹配 Memory 阶段写入寄存器：**
 - $\text{Match_1E_M} = (\text{RA1E} == \text{WA3M})$
 - $\text{Match_2E_M} = (\text{RA2E} == \text{WA3M})$
- **检查 Execute 阶段寄存器是否匹配 Writeback 阶段写入寄存器：**
 - $\text{Match_1E_W} = (\text{RA1E} == \text{WA3W})$
 - $\text{Match_2E_W} = (\text{RA2E} == \text{WA3W})$

前推选择逻辑 (ForwardAE)：[p.153]

```

if (Match_1E_M & RegWriteM) ForwardAE = 10; // 从Memory阶段前推
else if (Match_1E_W & RegWriteW) ForwardAE = 01; // 从Writeback阶段前推
else ForwardAE = 00; // 使用RF读取值

```

ForwardBE 的逻辑类似（使用 Match_2E_x）。[p.153]

前推优先级：Memory 阶段优先于 Writeback 阶段（更近的结果更“新鲜”）。[p.152]

4.3.3 停顿 (Stalling)

[p.154–157]

当一条 LDR 指令的结果被下一条指令使用时，即使有前推也无法在 Execute 阶段获得数据（LDR 的数据在 Memory 阶段结束时才从内存读出），必须停顿一个周期。

停顿逻辑：[p.157]

- 检查 Decode 阶段的任一源寄存器是否匹配 Execute 阶段正在写入的寄存器（且 Execute 阶段正在执行 LDR）：

$$\text{Match_12D_E} = (\text{RA1D} == \text{WA3E}) \vee (\text{RA2D} == \text{WA3E})$$

$$\text{ldrstall} = \text{Match_12D_E} \wedge \text{MemtoRegE}$$

- 停顿效果：

$$\text{StallF} = \text{StallD} = \text{FlushE} = \text{ldrstall}$$

Fetch 和 Decode 阶段暂停（stall），Execute 阶段被清空（flush，变成 NOP）。[p.157]

4.3.4 控制冒险 (Control Hazards)

[p.158–163]

B 指令的问题： [p.158]

- 分支目标地址在 Writeback 阶段才确定。
- 在分支发生前，已取出了 4 条后续指令。
- 如果分支发生（taken），这 4 条指令必须被清空（flush）。
- 对 PC（R15）的写入操作同理。[p.158]

分支误预测惩罚 (Branch Misprediction Penalty)： 当分支发生时，需要清空的指令数（= 4）。[p.159]

4.3.5 提前分支解析 (Early Branch Resolution)

[p.160–162]

通过在 Execute 阶段确定 BTA 来减少分支误预测惩罚：[p.160]

- 分支误预测惩罚从 4 个周期降到 **2 个周期**。[p.160]
- 硬件改动：[p.160]
 - 在 PC 寄存器前增加一个分支多路选择器，选择来自 ALUResultE 的 BTA。
 - 增加 BranchTakenE 选择信号（仅在分支条件满足时有效）。
 - PCSrcW 现在仅用于 PC 的写入（非分支的情况）。

4.3.6 控制停顿逻辑

[p.163]

完整的停顿/清空逻辑：

- **PCWrPendingF**：Decode、Execute 或 Memory 阶段是否有 PC 写操作待处理。

$$\text{PCWrPendingF} = \text{PCSrcD} + \text{PCSrcE} + \text{PCSrcM}$$

- **StallF** = ldrStallD + PCWrPendingF：取指阶段在有 PC 写待处理时停顿。[p.163]
- **FlushD** = PCWrPendingF + PCSrcW + BranchTakenE：分支发生或 PC 被写时清空译码阶段。[p.163]
- **FlushE** = ldrStallD + BranchTakenE：分支发生或 LDR 停顿发生时清空执行阶段。[p.163]
- **StallD** = ldrStallD。[p.163]

4.4 流水线处理器性能分析

[p.165–171]

4.4.1 流水线 CPI 计算示例

[p.165–166]

SPECINT2000 基准：25% loads, 10% stores, 13% branches, 52% DP。假设：[p.165]

- 40% 的 loads 被下一条指令使用 → 需要停顿。
- 50% 的分支被误预测 → 需要清空 2 条指令。

各指令的 CPI: [p.166]

- Load CPI = $1(0.6) + 2(0.4) = \mathbf{1.4}$ (不 stall 时为 1, stall 时为 2)
- Store CPI = **1**
- Branch CPI = $1(0.5) + 3(0.5) = \mathbf{2}$ (不 stall 时为 1, 误预测为 3)
- DP CPI = **1**

$$\text{Average CPI} = (0.25)(1.4) + (0.10)(1) + (0.13)(2) + (0.52)(1) = \mathbf{1.23}$$

4.4.2 流水线关键路径

[p.167–169]

关键路径受最慢的流水线阶段限制: [p.167]

$$T_{c3} = \max \begin{cases} t_{pcq} + t_{mem} + t_{setup} & \text{Fetch} \\ 2(t_{RFread} + t_{setup}) & \text{Decode} \\ t_{pcq} + 2t_{mux} + t_{ALU} + t_{setup} & \text{Execute} \\ t_{pcq} + t_{mem} + t_{setup} & \text{Memory} \\ 2(t_{pcq} + t_{mux} + t_{RFwrite}) & \text{Writeback} \end{cases}$$

使用相同的延迟参数 ($t_{RFread} = 100\text{ps}$, $t_{setup} = 50\text{ps}$, $t_{RFwrite} = 70\text{ps}$): [p.168–169]

$$T_{c3} = 2(t_{RFread} + t_{setup}) = 2[100 + 50] = \mathbf{300 \text{ ps}}$$

Decode 阶段 (两次寄存器读) 成为瓶颈。[p.169]

4.4.3 流水线执行时间

[p.170]

对于 1000 亿条指令的程序:

$$\text{Execution Time} = (100 \times 10^9)(1.23)(300 \times 10^{-12}) = \mathbf{36.9 \text{ 秒}}$$

4.4.4 三种处理器性能对比

[p.171]

处理器类型	执行时间 (s)	加速比
单周期 (Single-Cycle)	84	1
多周期 (Multicycle)	140	0.6
流水线 (Pipelined)	36.9	2.28

关键结论：流水线实现了近 2.3 倍的单周期加速（与单周期基准相比）。多周期实际上由于 CPI 开销比单周期更慢。[p.171]

5 高级微体系结构 (Advanced Microarchitecture)

5.1 深度流水线 (Deep Pipelining)

[p.173]

现代处理器通常有 10–20 级流水线阶段。深度流水线的限制因素：[p.173]

- **流水线冒险 (Pipeline Hazards)**: 更深流水线导致更大的误预测惩罚。[p.173]
- **时序开销 (Sequencing Overhead)**: 流水线寄存器的建立保持时间开销随级数增加而累积。[p.173]
- **功耗 (Power)**: 更多流水线寄存器意味着更多时钟功耗。[p.173]
- **成本 (Cost)**: 更多流水线寄存器增加芯片面积。[p.173]

5.2 微操作 (Micro-operations)

[p.174–175]

定义: 将复杂的指令分解为一系列简单的指令, 称为微操作 (**micro-ops**, μ -ops)。[p.174]

- 在运行时, 复杂指令被解码为一条或多条微操作。[p.174]
- 广泛用于 CISC 架构 (如 x86)。[p.174]
- ARM 也使用微操作处理某些指令, 例如: [p.174]

复杂指令	微操作序列
LDR R1, [R2], #4	LDR R1, [R2] ADD R2, R2, #4

- 如果没有微操作, 这种指令需要为寄存器文件额外增加一个写端口。[p.174]

ARM 的平衡策略: [p.175]

- 代码密度优于纯 RISC 指令集 (如 MIPS)。[p.175]
- 解码效率优于 CISC 指令集 (如 x86)。[p.175]

5.3 分支预测 (Branch Prediction)

[p.176–180]

5.3.1 静态分支预测

[p.176–177]

预测分支是否发生 (taken) 的策略:

- 向后分支 (**Backward Branch**) 通常被预测为 taken (循环)。[p.176]
- 向前分支 (**Forward Branch**) 通常被预测为 not taken。[p.177]
- 检查分支方向 (forward or backward): 若向后则预测 taken, 否则预测 not taken。
[p.177]

5.3.2 动态分支预测

[p.177–180]

动态预测器在分支目标缓冲区 (**Branch Target Buffer, BTB**) 中保存最近几百 (甚至几千) 条分支的历史信息: [p.177]

- 分支目标地址。[p.177]
- 分支是否被 taken。[p.177]

1 位分支预测器: [p.179]

- 记住上次分支是否发生, 本次预测相同结果。
- 缺点: 对循环的第一次和最后一次分支会误预测。[p.179]

2 位分支预测器 (4 状态有限状态机): [p.180]

- 四种状态: Strongly Taken → Weakly Taken → Weakly Not Taken → Strongly Not Taken。
- 需要连续两次误预测才会改变预测方向。
- 仅对循环的最后一次分支误预测 (比 1 位少一次误预测)。[p.180]

5.4 超标量处理器 (Superscalar)

[p.181–183]

定义: 包含多份数据通路副本, 能够在同一时钟周期内同时执行多条指令。[p.181]

- 理想 $IPC = 2$ (双发射), 但实际受限于指令间的依赖关系 (dependencies)。[p.182–183]
- 示例: 无依赖时实际 $IPC = 2$ 。[p.182]
- 示例: 有依赖时实际 $IPC = 6/5 = 1.2$ 。[p.183]
- 依赖关系使得同时发射多条指令变得困难。[p.181]

5.5 乱序处理器 (Out of Order Processor)

[p.184–186]

定义: 处理器可以在不违反数据依赖的前提下, 以不同于程序顺序的顺序发射指令。
[p.184]

三种数据依赖: [p.184]

- **RAW (Read After Write):** 一条指令写寄存器, 后续指令读同一寄存器—真依赖 (true dependency)。[p.184]
- **WAR (Write After Read):** 一条指令读寄存器, 后续指令写同一寄存器—反依赖 (anti-dependency)。[p.184]
- **WAW (Write After Write):** 两条指令都写同一寄存器—输出依赖 (output dependency)。[p.184]

指令级并行 (Instruction Level Parallelism, ILP): 可以同时发射的指令数, 通常平均 < 3 。[p.185]

记分板 (Scoreboard): 一个表, 跟踪: [p.185]

- 等待发射的指令。
- 可用的功能单元。
- 指令间的依赖关系。

乱序执行示例: [p.186]

```
LDR R8, [R0, #40]
ADD R9, R8, R1      ; 依赖R8
SUB R8, R2, R3      ; 写R8 (WAR依赖)
AND R10, R4, R8     ; 依赖R8 (新值)
ORR R11, R5, R6     ; 无依赖
STR R7, [R11, #80]  ; 依赖R11
```

理想 $IPC = 2$, 实际 $IPC = 6/4 = 1.5$ 。[p.186]

5.6 寄存器重命名 (Register Renaming)

[p.187]

通过重命名消除 WAR 和 WAW 的伪依赖 (false dependency), 进一步提高 ILP。[p.187]
同一示例使用寄存器重命名后:

```
LDR R8, [R0, #40]
ADD R9, R8, R1
SUB R12, R2, R3    ; 重命名R8为R12, 消除WAR依赖
AND R10, R4, R12   ; 使用新名R12
ORR R11, R5, R6
STR R7, [R11, #80]
```

实际 $IPC = 6/3 = 2$ (达到理想值)。[p.187]

5.7 SIMD (单指令多数据)

[p.188]

定义: 单指令多数据 (Single Instruction Multiple Data), 一条指令同时对多个数据元素执行相同操作。[p.188]

- 常见应用: 图形处理 (graphics)。[p.188]
- 执行短算术操作 (也称为 packed arithmetic/打包算术)。[p.188]
- 例如: 一条指令同时对八个 8 位元素做加法。[p.188]

5.8 多线程 (Multithreading)

[p.189–192]

5.8.1 线程定义

[p.190]

- **进程 (Process):** 在计算机上运行的程序。多个进程可以同时运行 (如浏览网页、听音乐、写论文)。[p.190]
- **线程 (Thread):** 程序的一部分。每个进程可以有多个线程 (如文字处理程序可能有输入线程、拼写检查线程、打印线程)。[p.190]

5.8.2 传统处理器中的线程

[p.191]

- 一次只能运行一个线程。[p.191]
- 当当前线程停顿时（如等待内存），进行上下文切换 (**Context Switching**):
 - 保存当前线程的架构状态。
 - 加载等待线程的架构状态。
 - 运行等待线程。
- 对于用户来说，感觉所有线程在同时运行。[p.191]

5.8.3 硬件多线程

[p.192]

- 维护多份架构状态（多套寄存器文件）。[p.192]
- 多个线程同时活跃：
 - 当一个线程停顿时，另一个线程可以立即运行（无需 OS 级别的上下文切换开销）。[p.192]
 - 如果一个线程不能充分利用所有执行单元，另一个线程可以使用空闲的执行单元。[p.192]
- 不增加单线程的 ILP，但增加了总体吞吐量 (**throughput**)。[p.192]
- Intel 将此技术称为”超线程 (**Hyperthreading**)”。[p.192]

5.9 多处理器 (Multiprocessors)

[p.193]

定义：多个处理器（核心）通过某种通信机制协同工作。[p.193]

三种类型：[p.193]

- 同构 (**Homogeneous**)：多个相同核心共享主存。如多核 CPU。[p.193]
- 异构 (**Heterogeneous**)：不同核心负责不同任务。如手机中的 DSP + CPU。[p.193]
- 集群 (**Clusters**)：每个核心有自己的存储系统。[p.193]

5.10 延伸阅读资源

[p.194]

- Patterson & Hennessy: *Computer Architecture: A Quantitative Approach*. [p.194]
- 重要会议: ISCA (International Symposium on Computer Architecture)、HPCA (International Symposium on High Performance Computer Architecture)。 [p.194]

6 核心公式速查表 (Key Formula Quick Reference)

公式	页码	说明
$\text{Execution Time} = \#I \times \text{CPI} \times T_c$	p.5	处理器性能基本公式
$\text{BTA} = \text{ExtImm} + (\text{PC} + 8)$	p.26	分支目标地址计算
$T_{c1} = 840 \text{ ps}$	p.76	单周期关键路径延迟
$\text{CPI (multicycle)} = 4.12$	p.129	多周期平均 CPI
$T_{c2} = 340 \text{ ps}$	p.132	多周期关键路径延迟
$\text{CPI (pipelined)} = 1.23$	p.166	流水线平均 CPI
$T_{c3} = 300 \text{ ps}$	p.169	流水线关键路径延迟
单周期 ET = 84 s	p.77	100B 指令单周期执行时间
多周期 ET = 140 s	p.134	100B 指令多周期执行时间
流水线 ET = 36.9 s	p.170	100B 指令流水线执行时间

7 三种实现对比总结

特性	单周期	多周期	流水线
时钟周期	最长 (840 ps)	中等 (340 ps)	最快 (300 ps)
CPI	1	4.12	1.23
执行时间 (100B 指令)	84 s	140 s	36.9 s
复杂度	最简单	中等	最复杂
硬件复用	无	有	有
每条指令周期数	固定	可变	1 (理想)
主要瓶颈	关键路径长	CPI 开销大	冒险处理

8 考点指导 (Study Guidance)

8.1 高频考点

- 三种微体系结构（单周期/多周期/流水线）的优缺点对比。[p.4, 78–80, 138]
- 单周期数据通路的构建过程（以 LDR 为例，6 步构建）。[p.11–17]

- 主解码器真值表（控制信号的含义和赋值）。[p.40]
- ALU 解码器（FlagW 信号与指令 S 位的关系）。[p.45]
- 条件码表（14 种条件助记符的含义）。[p.55]
- 条件执行与标志位更新逻辑。[p.50–63]
- 多周期 FSM 的状态转换和各指令周期数。[p.109–127]
- 数据前推（Forwarding）的条件判断逻辑。[p.149–153]
- 停顿逻辑（ldrstall 的推导）。[p.157]
- 分支误预测惩罚计算。[p.159–160]
- 性能计算：给定延迟参数，计算关键路径和执行时间。[p.75–77, 131–134, 165–170]
- 2 位分支预测器的状态机。[p.180]

8.2 中等考点

- PC 逻辑（PCS 的推导）。[p.47]
- ImmSrc 的三种立即数扩展方式。[p.27]
- 多周期条件逻辑与单周期的区别。[p.123]
- 提前分支解析（Early Branch Resolution）原理。[p.160]
- 控制停顿逻辑（StallF, FlushD, FlushE）。[p.163]
- 超标量、乱序执行、寄存器重命名的概念。[p.181–187]

8.3 了解性考点

- CMP 指令的实现（NoWrite 信号）。[p.66–69]
- 移位寄存器的数据通路扩展。[p.70–71]
- 深度流水线的限制因素。[p.173]
- 微操作（ μ -ops）的概念和 ARM 的平衡策略。[p.174–175]
- SIMD 的概念。[p.188]
- 多线程与多处理器的类型和区别。[p.189–193]